



# DATA MINING

## Complete Study Notes

All 10 Chapters | Visual Diagrams | Easy Language

KDD • Preprocessing • Warehousing • Association Rules • Classification • Clustering

1

Introduction to Data Mining

2

Data Preprocessing

3

Data Warehousing & OLAP

4

Association Rule Mining

5

Classification Techniques

6

Clustering Techniques

7

Outlier Analysis

8

Sequential Pattern Mining

9

Web Mining

10

Text Mining & Big Data

# 1

## Introduction to Data Mining

Data Mining Study Notes

### ■ What is Data Mining?

Data Mining is the process of **discovering useful patterns, rules, and insights** from large datasets. Think of it like digging for gold in a mountain of raw data!

### ■ KDD Process (Knowledge Discovery in Databases)

KDD is the overall process — Data Mining is just one step inside it.



| Step              | What happens?                                          |
|-------------------|--------------------------------------------------------|
| 1. Data Selection | Choose relevant data from databases                    |
| 2. Preprocessing  | Clean data — remove errors, fill missing values        |
| 3. Transformation | Convert data to suitable format (normalize, aggregate) |
| 4. Data Mining    | Apply algorithms to find patterns                      |
| 5. Evaluation     | Assess if patterns are useful and valid                |
| 6. Knowledge      | Present findings to decision makers                    |

### ■ Data Mining vs Data Warehousing

| Data Mining                  | Data Warehousing                   |
|------------------------------|------------------------------------|
| Discovers patterns & rules   | Stores & organizes historical data |
| Uses algorithms (ML, stats)  | Uses ETL processes                 |
| Output: patterns/models      | Output: reports/queries            |
| Predictive & descriptive     | Mainly descriptive                 |
| E.g. finding customer trends | E.g. storing sales history         |

### ■ OLTP vs OLAP

| Feature | OLTP                    | OLAP                 |
|---------|-------------------------|----------------------|
| Purpose | Day-to-day transactions | Analysis & reporting |

|            |                        |                          |
|------------|------------------------|--------------------------|
| Data       | Current, detailed      | Historical, aggregated   |
| Operations | INSERT, UPDATE, DELETE | SELECT (complex queries) |
| Speed      | Fast (milliseconds)    | Slower (seconds/minutes) |
| Users      | Clerks, staff          | Managers, analysts       |
| Example    | Bank ATM transaction   | Monthly sales report     |
| DB Size    | GB range               | TB range                 |

#### Quick Memory Tips

- OLTP = Online Transaction Processing (operational)
- OLAP = Online Analytical Processing (analytical)
- Data Warehouses are designed for OLAP workloads

## 2

## Data Preprocessing

Data Mining Study Notes

Raw data is almost always **dirty, incomplete, or inconsistent**. Preprocessing cleans and prepares it. Think of it like washing vegetables before cooking!

### ■ Step 1: Data Cleaning

#### Handling Missing Values

| Method            | When to use                        | Example                             |
|-------------------|------------------------------------|-------------------------------------|
| Mean Imputation   | Numeric data, no outliers          | Age avg = 30, fill missing with 30  |
| Median Imputation | Numeric data WITH outliers         | Salary median is robust to extremes |
| Mode Imputation   | Categorical data                   | Most common city = 'Delhi'          |
| Regression        | When other features can predict it | Predict missing income from age/job |
| Delete Row        | Very few rows missing              | If <5% rows affected                |

#### Noise Removal Techniques

| Technique  | How it works                                                                |
|------------|-----------------------------------------------------------------------------|
| Binning    | Sort data, divide into equal-frequency bins, smooth by mean/median/boundary |
| Regression | Fit a regression line, use predicted values to smooth noise                 |
| Clustering | Group similar data; detect and remove outlier clusters                      |

##### Binning Example:

Sorted: 4, 8, 9 | 15, 21, 24 | 25, 28, 34

Bin1(mean=7): 7,7,7 | Bin2(mean=20): 20,20,20 | Bin3(mean=29): 29,29,29

**Result: Smoother data with less noise!**

#### Outlier Detection

- IQR Method: Values  $< Q1 - 1.5 * IQR$  or  $> Q3 + 1.5 * IQR$  are outliers
- Z-score:  $|z| > 3$  typically indicates an outlier
- Box plots: Visual way to spot outliers
- Clustering: Points far from any cluster = outliers

### ■ Step 2: Data Integration

Combining data from **multiple sources** into a unified view.

- Schema Integration: Map attributes from different DBs (e.g., cust\_id vs customer\_id)
- **Entity Identification Problem**: Is 'Jon Smith' in DB1 the same as 'John Smith' in DB2?
- Redundancy Detection: Use correlation to find redundant attributes
- Data Value Conflicts: Same entity, different values (units, formats)

## ■ Step 3: Data Transformation

### Normalization Methods

| Method                 | Formula                               | Range    | Best for                  |
|------------------------|---------------------------------------|----------|---------------------------|
| Min-Max                | $(x - \min) / (\max - \min)$          | [0, 1]   | When exact range is known |
| Z-score                | $(x - \text{mean}) / \text{std\_dev}$ | ~[-3, 3] | When outliers exist       |
| <b>Decimal Scaling</b> | $x / 10^j$ (j = digits in max)        | [-1, 1]  | Simple integer data       |

### Other Transformations

- Aggregation: Combine fine-grained data (daily -> monthly)
- **Smoothing**: Remove noise **using binning/regression**
- Generalization: Replace low-level data with high-level (25 years -> 'young adult')
- Attribute construction: Create new attributes from existing ones

## ■ Step 4: Data Reduction

Reduce the volume of data while keeping analytical results the same.

### Dimensionality Reduction

- PCA (Principal Component Analysis): Finds new axes (principal components) that capture maximum variance. Reduces many
- Feature Selection: Keep only the most relevant features (forward/backward selection)
- Attribute Subset Selection: Remove irrelevant/redundant attributes

### Numerosity Reduction

- **Parametric**: Fit data to a model (regression), store only model parameters
- Non-parametric: Histograms, clustering, sampling
- Sampling: Use a representative subset (random, stratified, cluster sampling)

| Sampling Type | How it works                                 |
|---------------|----------------------------------------------|
| Simple Random | Each item has equal probability of selection |

|            |                                                                    |
|------------|--------------------------------------------------------------------|
| Stratified | Divide into groups (strata), sample from each group proportionally |
| Cluster    | Divide into clusters, randomly select entire clusters              |
| Systematic | Pick every k-th item from sorted list                              |

# 3

## Data Warehousing & OLAP

Data Mining Study Notes

A **Data Warehouse** is a central repository of integrated data from multiple sources, designed for analysis and reporting.

### ■ Data Warehouse Architecture

| Layer              | Description                                              |
|--------------------|----------------------------------------------------------|
| Source Layer       | Operational databases, flat files, external data sources |
| Staging Layer      | Temporary area for data cleaning and transformation      |
| Data Warehouse     | Central integrated repository (star/snowflake schema)    |
| Data Marts         | Subject-specific subsets (sales mart, HR mart)           |
| Presentation Layer | OLAP tools, dashboards, reports                          |

### ■ ETL vs ELT Process

| ETL (Extract, Transform, Load) | ELT (Extract, Load, Transform)    |
|--------------------------------|-----------------------------------|
| Traditional approach           | Modern cloud approach             |
| Transform BEFORE loading       | Transform AFTER loading           |
| Good for structured data       | Good for big/unstructured data    |
| Staging area used              | Target system does transformation |
| SQL Server, Oracle DW          | Snowflake, BigQuery, Redshift     |

### ■ OLAP Operations

OLAP lets analysts view data from multiple **dimensions** (time, region, product, etc.)

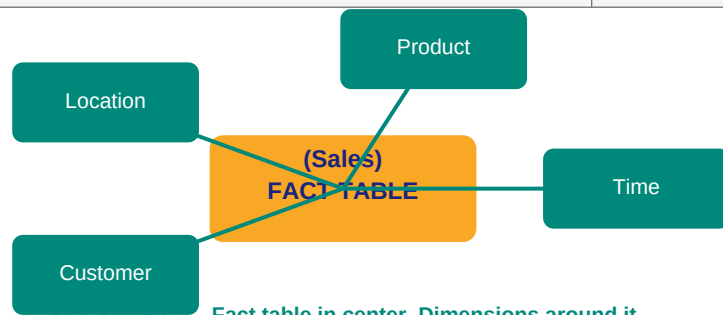
| Operation  | What it does                                      | Memory trick                       |
|------------|---------------------------------------------------|------------------------------------|
| Roll-up    | Aggregate to higher level (day → month → year)    | Zoom OUT, see bigger picture       |
| Drill-down | Go to lower level detail (year → month → day)     | Zoom IN, see more detail           |
| Slice      | Select one value of a dimension (e.g., year=2024) | Cut a thin slice of the cube       |
| Dice       | Select range on multiple dimensions               | Cut a small cube from the big cube |
| Pivot      | Rotate the cube to view from different angle      | Turn the table/spreadsheet         |

### ■ Data Cube

- Data Cube is an n-dimensional data model for OLAP analysis
- 3D example: Axes = Product, Time, Location
- Each cell = measure value (e.g., sales amount)
- **Cuboids: all possible aggregations of the cube dimensions**
- **Apex cuboid: fully aggregated** (total sales across all dimensions)

## ■ Schema Types

| Schema                  | Structure                                             | Pros/Cons                             |
|-------------------------|-------------------------------------------------------|---------------------------------------|
| <b>Star</b> Schema      | One central fact table surrounded by dimension tables | Simple, fast queries; some redundancy |
| <b>Snowflake</b> Schema | Normalised dimension tables (hierarchies split out)   | Less redundancy; more complex joins   |
| Fact Constellation      | Multiple fact tables sharing dimension tables         | Flexible; complex to manage           |



Star Schema — Fact table in center, Dimensions around it

# 4

## Association Rule Mining

Data Mining Study Notes

Association rules find items that often appear **together**. Classic example: "People who buy diapers also buy beer!" (Market basket analysis)

### Key Definitions

- Itemset: A set of items e.g. {Bread, Butter, Milk}
- Frequent Itemset: Itemset whose support  $\geq$  min\_support threshold
- Association Rule:  $X \Rightarrow Y$  (If you buy X, you also buy Y)

### Core Measures

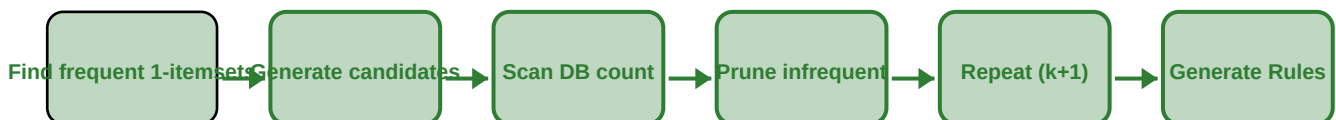
| Measure    | Formula                                                                                 | Meaning                             |
|------------|-----------------------------------------------------------------------------------------|-------------------------------------|
| Support    | $\text{support}(X \cup Y) = \text{count}(X \cup Y) / N$                                 | How often X and Y appear together   |
| Confidence | $\text{conf}(X \Rightarrow Y) = \text{support}(X \cup Y) / \text{support}(X)$           | Given X, probability of Y           |
| Lift       | $\text{lift}(X \Rightarrow Y) = \frac{\text{conf}(X \Rightarrow Y)}{\text{support}(Y)}$ | How much more likely than by chance |

#### Interpreting Lift

- Lift  $> 1$ : Positive association (useful rule)
- Lift  $= 1$ : Independent (useless rule)
- Lift  $< 1$ : Negative association

### Apriori Algorithm

Works on the **Anti-monotone property**: If an itemset is infrequent, ALL its supersets are also infrequent (prune them!).



### Apriori Steps

- Step 1: Scan DB to find all frequent 1-itemsets (L1)
- Step 2: Join L<sub>k-1</sub> with itself to generate candidate C<sub>k</sub>
- Step 3: Prune candidates with any infrequent subset
- Step 4: Scan DB to count candidates, keep frequent ones
- Step 5: Repeat until no new frequent itemsets found
- Limitation: Multiple DB scans, memory for candidates

## ■ FP-Growth Algorithm

More efficient than Apriori — **no candidate generation!** Uses a compressed tree structure (FP-Tree).

### FP-Growth Steps

- Step 1: Scan DB once, find frequent items, sort by frequency
- Step 2: Scan DB again, build FP-Tree (compressed representation)
- Step 3: Mine FP-Tree using conditional pattern bases
- No candidate generation → much faster than Apriori
- Only 2 DB scans (vs multiple in Apriori)

| Apriori                   | FP-Growth                 |
|---------------------------|---------------------------|
| Multiple DB scans         | Only 2 DB scans           |
| Generates many candidates | No candidate generation   |
| Slow on large datasets    | Faster on large datasets  |
| Easy to understand        | More complex to implement |
| Good for small data       | Better for large data     |

## ■ Types of Association Rules

| Type               | Description                                                     |
|--------------------|-----------------------------------------------------------------|
| Boolean Rules      | Items either present (1) or absent (0): Bread => Butter         |
| Quantitative Rules | Involve numeric attributes: Age(30-40) => Salary(60K+)          |
| Single-level       | All items at same concept level: Milk => Bread                  |
| Multi-level        | Items at different levels: Dairy => Bakery (broader categories) |

## ■ Interestingness Measures

- Subjective: Interesting if unexpected or actionable for the user
- Objective: Based on statistical measures (support, confidence, lift)
- Correlation analysis: chi-square test to check if X and Y are statistically related
- Rules with high support+confidence but low lift are NOT interesting!

# 5

## Classification Techniques

Data Mining Study Notes

Classification assigns a **class label** to new data based on a model learned from training data. Example: Is this email spam or not spam?

### Overfitting & Underfitting

| Problem      | What it means                                                                                | Fix                                                  |
|--------------|----------------------------------------------------------------------------------------------|------------------------------------------------------|
| Overfitting  | Model memorizes training data, fails on new data. High training accuracy, low test accuracy. | Pruning, cross-validation, more data, regularization |
| Underfitting | Model too simple, misses patterns. Low accuracy on both training and test data.              | More features, complex model, longer training        |
| Good fit     | Model generalises well to unseen data.                                                       | Bias-variance tradeoff balanced!                     |

### Decision Trees

A tree where each **internal node** tests an attribute, each **branch** is an outcome, and each **leaf** is a class label.

| Algorithm | Splitting Criterion                                   | Key Feature                                                     |
|-----------|-------------------------------------------------------|-----------------------------------------------------------------|
| ID3       | Information Gain (entropy)                            | Only <b>categorical</b> attributes; <b>no pruning</b>           |
| C4.5      | Gain Ratio (corrects bias towards multi-valued attrs) | Handles <b>continuous</b> , missing values; <b>post-pruning</b> |
| CART      | <b>Gini Index</b>                                     | <b>Binary splits only</b> ; both classification & regression    |

#### Key Formulas

- $\text{Entropy}(S) = -\sum(p_i \cdot \log_2(p_i))$  [0=pure, 1=max impure]
- $\text{Information Gain} = \text{Entropy}(\text{parent}) - \text{weighted sum of child entropies}$
- $\text{Gain Ratio} = \text{Info Gain} / \text{Split Info}$  (penalizes high-branch attrs)
- $\text{Gini Index} = 1 - \sum(p_i^2)$  [0=pure, 0.5=max impure for binary]

### Naive Bayes Classifier

Based on **Bayes' Theorem**. Assumes all features are *independent* given the class (naive assumption).

$P(C|X) = P(X|C) \cdot P(C) / P(X) \rightarrow$  Pick class C with highest posterior probability

- Fast, works well even with small data
- Good for text classification (spam filtering)
- Assumption of independence rarely holds in reality, but still performs well!
- Handles missing values naturally (skip in calculation)

## ■ k-Nearest Neighbours (k-NN)

Classify a new point by looking at the **k closest training points** and taking majority vote.

- Choose k (e.g. k=3, k=5): odd k avoids ties
- Distance measure: usually Euclidean distance
- Lazy learner: no training phase — all computation at test time
- High k: smoother boundary (less overfit), low k: complex boundary
- Curse of dimensionality: performance degrades with many features

## ■ Support Vector Machine (SVM)

Find the **optimal hyperplane** that maximises the margin between classes.

- Support Vectors: Data points closest to the decision boundary
- Margin: Distance between support vectors; SVM maximises this
- Kernel trick: Map data to higher dimensions for non-linear classification (RBF, polynomial)
- Works well with high-dimensional data (text classification)
- Effective even when features > samples

|                  |          | Prediction Metrics    |                       |
|------------------|----------|-----------------------|-----------------------|
| Confusion Matrix |          | Predicted +           | Predicted -           |
|                  | Actual + | (True Positive)<br>TP | (Type II Error)<br>FN |
|                  | Actual - | (Type I Error)<br>FP  | (True Negative)<br>TN |

| Metric    | Formula                 | Meaning                                                |
|-----------|-------------------------|--------------------------------------------------------|
| Accuracy  | $(TP+TN)/(TP+TN+FP+FN)$ | Overall correctness                                    |
| Precision | $TP/(TP+FP)$            | Of predicted positive, how many are actually positive? |

|                      |                                  |                                               |
|----------------------|----------------------------------|-----------------------------------------------|
| Recall (Sensitivity) | $TP/(TP+FN)$                     | Of actual positives, how many did we catch?   |
| F1-Score             | $2 * Precision * Recall / (P+R)$ | Harmonic mean of Precision and Recall         |
| Type I Error (FPR)   | $FP/(FP+TN)$                     | False Alarm rate — said yes when answer is no |
| Type II Error (FNR)  | $FN/(FN+TP)$                     | Miss rate — said no when answer is yes        |

### ROC Curve Interpretation

- ROC Curve: Plots TPR (Recall) vs FPR at various thresholds
- AUC (Area Under Curve): Closer to 1 = better classifier, 0.5 = random guess
- High precision but low recall: Missing many positives (conservative model)
- High recall but low precision: Many false alarms (aggressive model)

# 6

## Clustering Techniques

Data Mining Study Notes

Clustering groups data points so that **similar points are in the same cluster** and dissimilar points are in different clusters. No class labels needed (unsupervised)!

### Types of Clustering

| Type          | Approach                                    | Examples                |
|---------------|---------------------------------------------|-------------------------|
| Partitioning  | Divide data into k non-overlapping clusters | k-Means, k-Medoids      |
| Hierarchical  | Build a tree of clusters (dendrogram)       | Agglomerative, Divisive |
| Density-based | Clusters = dense regions of points          | DBSCAN                  |
| Grid-based    | Quantize space into grid cells              | STING, WaveCluster      |
| Model-based   | Fit data to a model distribution            | EM, Gaussian Mixture    |

### k-Means Algorithm



#### k-Means Properties

- Time complexity:  $O(n * k * d * \text{iterations})$  — efficient!
- Sensitive to initial centroid choice (use k-means++ for better init)
- Sensitive to outliers (outliers can pull centroids)
- Assumes spherical clusters of similar size
- Choose k using Elbow Method (plot inertia vs k)

### k-Medoids (PAM Algorithm)

Like k-Means, but cluster center = an **actual data point** (medoid), not the mean.

| k-Means                  | k-Medoids                             |
|--------------------------|---------------------------------------|
| Center = mean of cluster | Center = actual data point (medoid)   |
| Sensitive to outliers    | Robust to outliers                    |
| Faster computation       | Slower ( $O(k(n-k)^2)$ per iteration) |
| Any numeric data         | Works with any distance measure       |
| Not robust to noise      | More robust to noise                  |

## ■ Hierarchical Clustering

- Agglomerative (bottom-up): Start with each point as cluster, merge closest clusters
- Divisive (top-down): Start with all points in one cluster, split recursively
- Result shown as **Dendrogram** (tree diagram)
- Cut dendrogram at chosen height to get desired number of clusters
- Linkage methods: Single (min distance), Complete (max), Average, Ward (minimize variance)

## ■ ■ DBSCAN

**Density-Based Spatial Clustering of Applications with Noise.** Finds clusters as **dense regions**.

- Parameters: **eps** (neighbourhood radius), **minPts** (minimum points in neighbourhood)
- Core Point: Has at least minPts neighbours within eps radius
- Border Point: Within eps of a core point but fewer than minPts neighbours
- Noise Point: Neither core nor border — treated as OUTLIERS
- Can find **arbitrarily shaped clusters**!
- Does NOT require specifying k in advance

## ■ Distance Measures

| Measure   | Formula                               | Used for                                                                 |
|-----------|---------------------------------------|--------------------------------------------------------------------------|
| Euclidean | $\sqrt{\sum((x_i - y_i)^2)}$          | Standard straight-line distance                                          |
| Manhattan | $\sum( x_i - y_i )$                   | Grid-like paths, city blocks                                             |
| Minkowski | $(\sum( x_i - y_i ^p))^{1/p}$         | Generalization: $p=1 \rightarrow$ Manhattan, $p=2 \rightarrow$ Euclidean |
| Cosine    | $\cos(\theta) = A \cdot B / ( A  B )$ | Text data, direction matters more than magnitude                         |

## 7

## Outlier Analysis

Data Mining Study Notes

Outliers are data points that **deviate significantly** from the majority of the data. They can be errors OR rare but important events!

### ■ Types of Outliers

| Type               | Definition                                   | Example                                               |
|--------------------|----------------------------------------------|-------------------------------------------------------|
| Global Outlier     | Deviates from the entire dataset             | A salary of \$10M in a dataset where average is \$50K |
| Contextual Outlier | Outlier only in a specific context/condition | 30°C is normal in summer but outlier in winter        |
| Collective Outlier | A group of points that are outliers together | Sudden spike in network traffic (DDoS attack)         |

### ■ ■ Detection Methods

| Method         | How                                                                                   | Pros/Cons                                                |
|----------------|---------------------------------------------------------------------------------------|----------------------------------------------------------|
| Statistical    | Assumes data follows a distribution; outliers = low probability points (z-score, IQR) | Simple, interpretable; assumes known distribution        |
| Distance-based | Points far from k-nearest neighbours are outliers                                     | No distribution assumption; slow for large data $O(n^2)$ |
| Density-based  | LOF (Local Outlier Factor): compare local density of point to its neighbours          | Handles varying density; parameter sensitive             |

#### Key Formulas & Rules

- IQR Method:  $Q1 - 1.5 \cdot IQR$  or  $Q3 + 1.5 \cdot IQR$  are thresholds
- Z-score:  $(x - \text{mean})/\text{std}$ ;  $|z| > 3$  is typically an outlier
- LOF score: 1 = normal;  $>> 1$  = outlier (much sparser than neighbours)
- Isolation Forest: Outliers are easier to isolate (shorter path in random trees)

# 8

## Sequential Pattern Mining

Data Mining Study Notes

Sequential pattern mining finds **frequent sub-sequences** in ordered/time-stamped data. Example: Customers who buy a phone, then buy a case, then buy earphones.

### ■ Key Concepts

- Sequence Database: Collection of sequences (each row = a customer's purchase history over time)
- Sequential Pattern: A sub-sequence that appears in at least min\_support sequences
- Example sequence: <{bread}{butter, milk}{cheese}> (time-ordered transactions)
- Length of sequence = total number of items in it

### ■ Algorithms

| Algorithm                             | Approach                                                            | Key Feature                                             |
|---------------------------------------|---------------------------------------------------------------------|---------------------------------------------------------|
| GSP (Generalised Sequential Patterns) | Apriori-like: generate candidates, scan DB, prune                   | Multiple DB scans, supports time constraints & taxonomy |
| PrefixSpan                            | Prefix-based projection: mine patterns without candidate generation | Much faster than GSP, uses projected databases          |

| GSP                               |
|-----------------------------------|
| Candidate generation like Apriori |
| Multiple DB scans                 |
| Handles time constraints          |
| Older, slower                     |

| PrefixSpan                  |
|-----------------------------|
| No candidate generation     |
| Projects DB for each prefix |
| Simpler, more efficient     |
| Faster on dense data        |

Web Mining applies data mining to extract useful knowledge from the World Wide Web — content, structure, and user behaviour.

### ■ Three Types of Web Mining

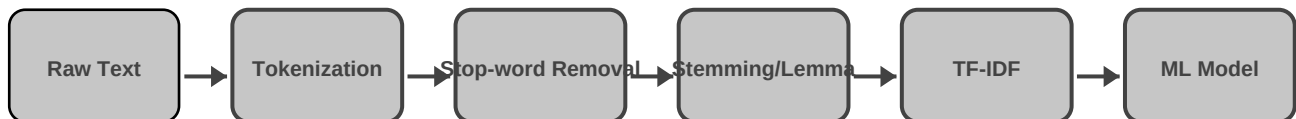
| Type                        | What it mines                                    | Examples                                     | Techniques                                         |
|-----------------------------|--------------------------------------------------|----------------------------------------------|----------------------------------------------------|
| Web <b>Content</b> Mining   | Text, images, audio, video on web pages          | Product info extraction, news classification | NLP, text mining, information extraction           |
| Web <b>Structure</b> Mining | Hyperlinks between web pages (graph structure)   | Google PageRank, HITS algorithm              | Graph analysis, link analysis                      |
| Web <b>Usage</b> Mining     | Server logs, clickstream data (how users browse) | Recommendation systems, personalisation      | Association rules, clustering, sequential patterns |

### ■ Web Structure Mining Algorithms

- **PageRank**: Assigns importance based on how many other important pages link to a page
- **HITS (Hyperlink-Induced Topic Search)**: Identifies Hubs (link to many good pages) and Authorities (linked by many hubs)
- Web Graph: Nodes = pages, Edges = hyperlinks

Text mining extracts structured information from unstructured text. Big data techniques scale data mining to massive datasets.

## ■ Text Preprocessing Pipeline



| Tokenization      | Split text into tokens (words/sentences)                      | "I love data" → ["I","love","data"] |
|-------------------|---------------------------------------------------------------|-------------------------------------|
| Stop-word Removal | Remove common words that carry no meaning                     | Remove: the, is, at, which, on      |
| Stemming          | Cut suffix to get root form (rough, may not be real word)     | running → run, studies → studi      |
| Lemmatization     | Get actual dictionary root form (more accurate than stemming) | running → run, better → good        |
| Chunking          | Group words into phrases (noun phrases, verb phrases)         | "big red dog" → NP chunk            |

## ■ TF-IDF

Measures word importance in a document relative to a corpus.

$TF(t, d) = (\text{Freq of term } t \text{ in doc } d) / (\text{Total terms in } d)$

$IDF(t) = \log(N / df(t))$  where  $N$ =total docs,  $df(t)$ =docs containing  $t$

$TF-IDF(t, d) = TF(t, d) \times IDF(t)$  – high score = important & rare word

## ■ Recommender Systems

| Collaborative Filtering | Recommend based on similar users' preferences       | Netflix: users like you also watched...              |
|-------------------------|-----------------------------------------------------|------------------------------------------------------|
| Content-Based Filtering | Recommend items similar to what user liked before   | Spotify: songs similar to ones you played            |
| Hybrid                  | Combine both collaborative and content-based        | Amazon: uses both purchase history and similar users |
| Knowledge-Based         | Use explicit domain knowledge and user requirements | Financial advisor recommending products              |

## ■ Time-Series Data Mining

- Trend Analysis: Long-term upward/downward movement
- Seasonality: Recurring patterns at regular intervals (holiday sales spikes)
- Anomaly Detection: Unusual points in the time series
- DTW (Dynamic Time Warping): Measure similarity between sequences of different lengths
- Forecasting: ARIMA, LSTM, Prophet models

## ■ Spatial Data Mining

- Mines patterns from geospatial data (location-tagged data)
- Spatial Association Rules: Nearby hospitals have lower crime rates
- Spatial Clustering: Find clusters of earthquakes, disease outbreaks
- Spatial outliers: A hospital with unusually high mortality in its region

### ■ Quick Comparison: ID3 vs C4.5 vs CART

| Splitting criterion | Information Gain | Gain Ratio               | Gini Index                  |
|---------------------|------------------|--------------------------|-----------------------------|
| Attribute types     | Categorical only | Categorical + Continuous | Both                        |
| Missing values      | Not handled      | Handled                  | Surrogate splits            |
| Pruning             | None             | Post-pruning             | Cost-complexity pruning     |
| Tree type           | Multi-way        | Multi-way                | Binary only                 |
| Output              | Classification   | Classification           | Classification + Regression |

### ■ Quick Comparison: k-Means vs k-Medoids

| Cluster center      | Mean of points (virtual) | Actual data point (medoid)          |
|---------------------|--------------------------|-------------------------------------|
| Outlier sensitivity | Highly sensitive         | Robust to outliers                  |
| Computational cost  | Lower $O(nkt)$           | Higher $O(k(n-k)^2)$                |
| Distance measures   | Usually Euclidean        | Any distance measure                |
| When to use         | Large data, no outliers  | Small/medium data, outliers present |

|           |         |                     |
|-----------|---------|---------------------|
| Algorithm | k-Means | PAM, CLARA, CLARANS |
|-----------|---------|---------------------|

## ■ QUICK REFERENCE CHEAT SHEET

### Key Formulas at a Glance

|                                 |                                                    |
|---------------------------------|----------------------------------------------------|
| Support( $X \Rightarrow Y$ )    | $\text{count}(X \cup Y) / N$                       |
| Confidence( $X \Rightarrow Y$ ) | $\text{support}(X \cup Y) / \text{support}(X)$     |
| Lift( $X \Rightarrow Y$ )       | $\text{conf}(X \Rightarrow Y) / \text{support}(Y)$ |
| Entropy(S)                      | $-\sum p_i \times \log_2(p_i)$                     |
| Gini Index                      | $1 - \sum p_i^2$                                   |
| Accuracy                        | $(TP+TN) / (TP+TN+FP+FN)$                          |
| Precision                       | $TP / (TP+FP)$                                     |
| Recall                          | $TP / (TP+FN)$                                     |
| F1-Score                        | $2PR / (P+R)$                                      |
| Min-Max Norm                    | $(x - \min) / (\max - \min)$                       |
| Z-score Norm                    | $(x - \text{mean}) / \text{std}$                   |
| TF-IDF                          | $TF(t, d) \times \log(N/df(t))$                    |

### Algorithm Selection Guide

| Task                           | Best Algorithm            | Why                         |
|--------------------------------|---------------------------|-----------------------------|
| Association Rules (small data) | Apriori                   | Simple, interpretable       |
| Association Rules (large data) | FP-Growth                 | Faster, no candidates       |
| Classification (interpretable) | Decision Tree (C4.5/CART) | Visual, easy to explain     |
| Classification (text)          | Naive Bayes               | Fast, works well with text  |
| Classification (complex)       | SVM                       | Good high-dim performance   |
| Clustering (round clusters)    | k-Means                   | Fast and efficient          |
| Clustering (arbitrary shapes)  | DBSCAN                    | Handles noise and shape     |
| Clustering (outliers present)  | k-Medoids                 | Robust to outliers          |
| Dimensionality Reduction       | PCA                       | Removes correlated features |